The Google File System

TaeYoung Rhee, Doyoung Kim

Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung
SOSP 2003

Outline

01 Design Assumptions & Architecture 3–8

02 Consistency Model 9–10

03 System Interactions 11–14

04 Master Operation 15–19

05 Fault Tolerance & Diagnosis 20–22

06 Performance Measurements 23–28

07 Experience, Related Work & Conclusions 29–31

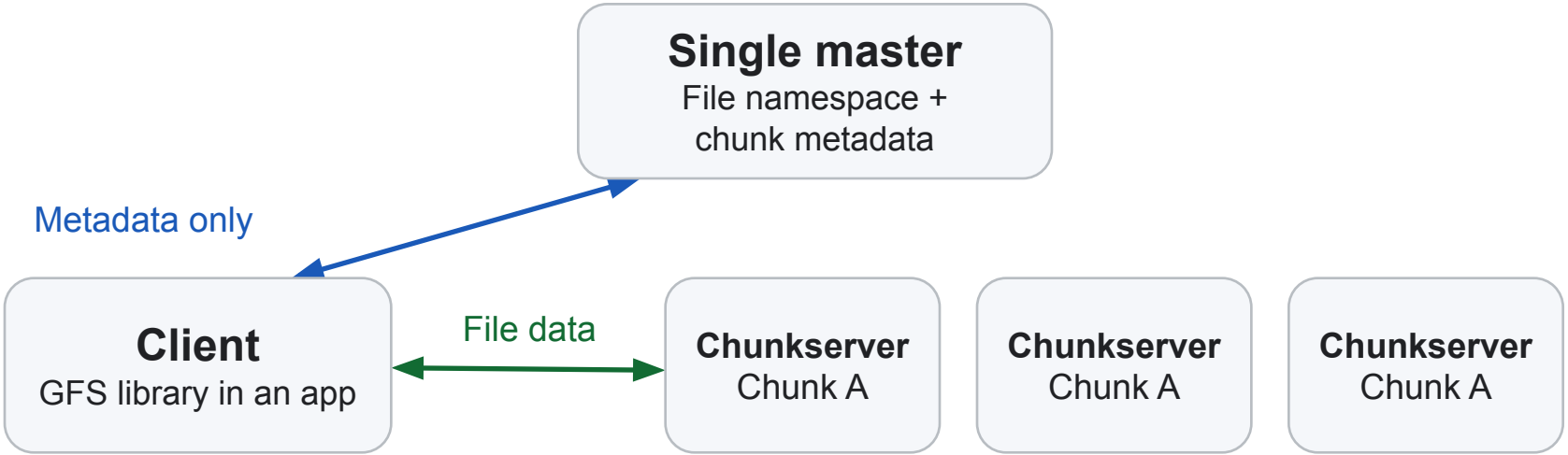
Design starts with the workload

- **Frequent failures** → monitoring, replication, automatic recovery
- **Large files and streaming reads** → favor sequential access
- **Append-heavy writes** → support concurrent record appends
- **Large batch jobs** → prioritize sustained bandwidth over latency
- **Application co-design** → tailor APIs and consistency guarantees

Source: Ghemawat et al., SOSP 2003, Sections 1 and 2.1

GFS architecture: three roles

One master manages metadata; chunkservers store replicated chunks.

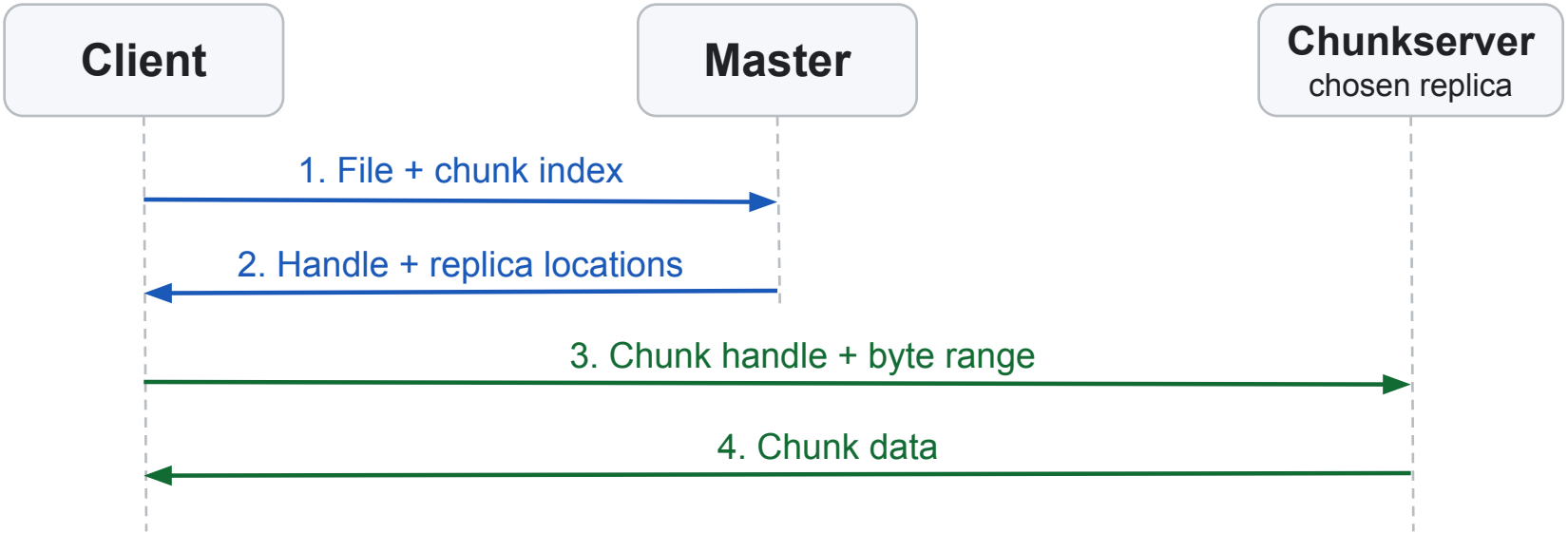


Example: one chunk, three replicas

Keep file data off the master.

Source: Sections 2.3–2.4 and Figure 1, Ghemawat et al., SOSP 2003

Read data bypasses the master

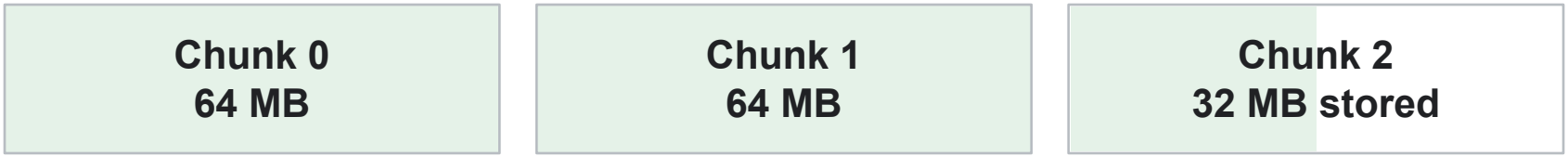


Cache locations to reduce master requests.
A single master simplifies metadata coordination.

Source: Section 2.4 and Figure 1, Ghemawat et al., SOSP 2003

Why 64 MB chunks?

Example: a 160 MB file = 64 MB + 64 MB + 32 MB



Chunk replicas grow only as needed; 64 MB is not preallocated.

Benefits

- Fewer master interactions
- Reuse TCP connections
- Less in-memory metadata

Tradeoff

Popular small files can concentrate read traffic.

Large chunks favor large, sequential workloads.

Source: Section 2.5, Ghemawat et al., SOSP 2003

Metadata stays in memory

Fast access to all three; different recovery paths.

Metadata	Example	Recovery source
Namespaces	/logs/run-1; chunk A	Log + checkpoint
File → chunk map	/logs/run-1 → [A, B]	Log + checkpoint
Replica locations	A → servers 2, 5, 9	Ask chunkservers

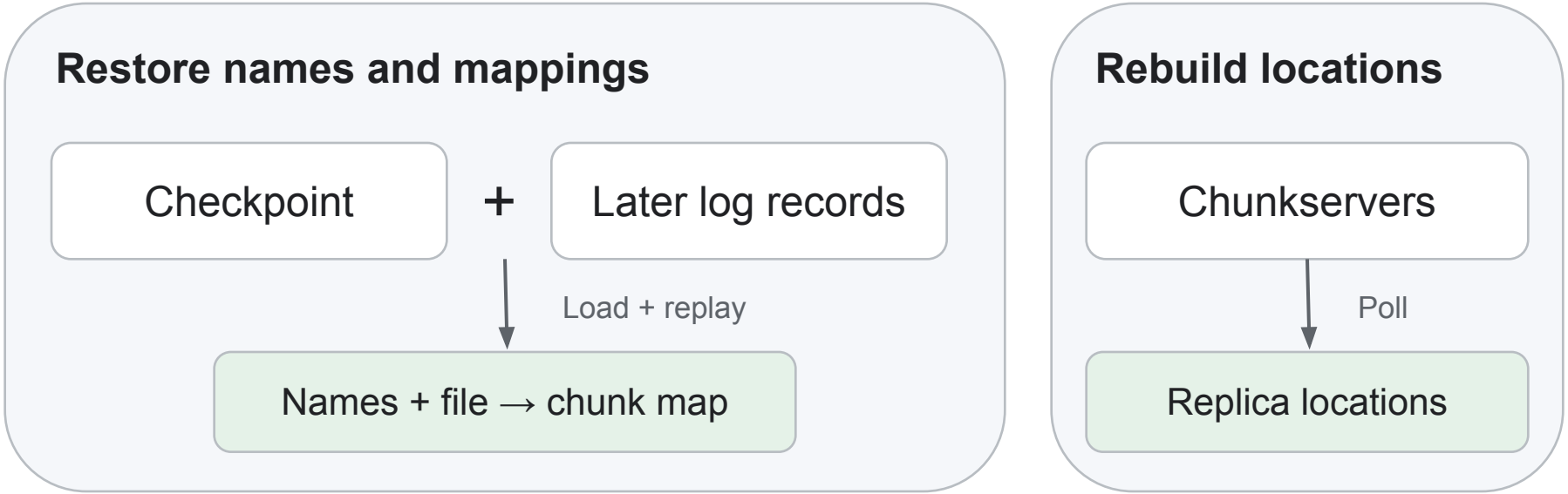
All three are in RAM; their recovery sources differ.

Paper reports <64 bytes of metadata per 64 MB chunk.

Source: Sections 2.6–2.6.2, Ghemawat et al., SOSP 2003

Recovering the master's metadata

Logical state is persisted; replica locations are discovered.



Flush log records to local and remote disks before reporting success.

Source: Sections 2.6.2–2.6.3, Ghemawat et al., SOSP 2003

Consistent vs. defined

These terms describe a file region after a mutation.

Illustration: application writes A = (A1, A2), B = (B1, B2)

Defined

R1: A1 A2
R2: A1 A2
R3: A1 A2

Same bytes;
a complete write

Consistent only

R1: A1 B2
R2: A1 B2
R3: A1 B2

Same bytes;
mixed fragments

Inconsistent

R1: A1 A2
R2: B1 B2
R3: A1 B2

Replicas disagree

Defined ⇒ consistent; consistent does not imply defined.

Illustrative fragments; definitions from Section 2.7.1, Ghemawat et al., SOSP 2003

Guarantees depend on the operation

Namespace changes are atomic. This table concerns file data.

Outcome	Write	Record append
Serial success	Defined	Defined regions interspersed with inconsistent regions
Concurrent successes	Consistent, but undefined	
Failure	Inconsistent	Inconsistent

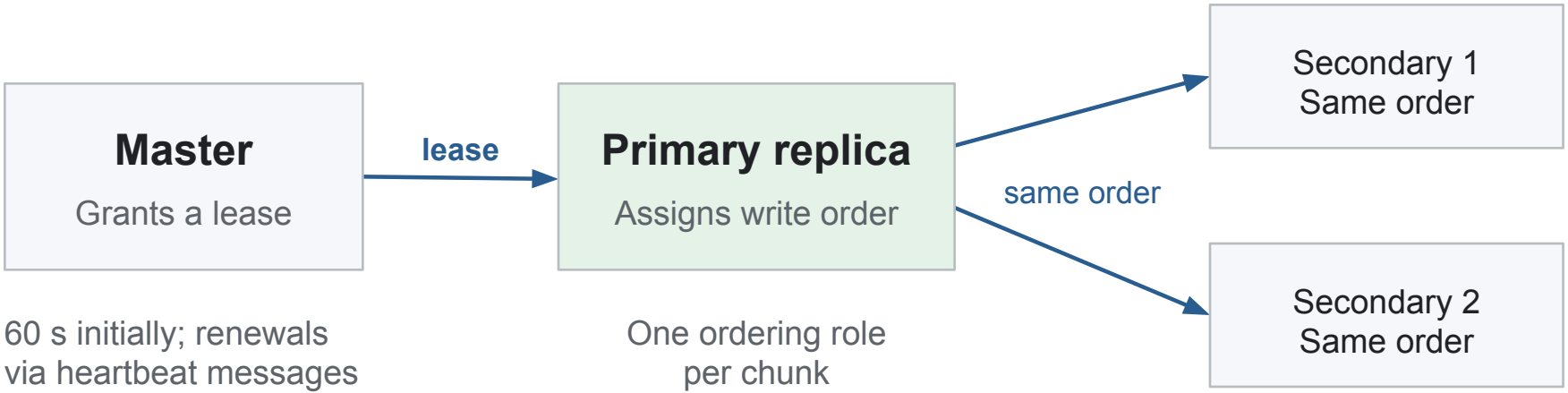
Write: client chooses offset. Record append: GFS chooses offset.

Successful append: atomic, at least once; duplicates are possible.

Source: Sections 2.7.1–2.7.2 and Table 1, Ghemawat et al., SOSP 2003

One primary orders mutations

The primary is one of the chunk's replicas, chosen by the master.



The master delegates ordering to the primary.

A new lease can be granted after the old lease expires.

Source: Section 3.1, Ghemawat et al., SOSP 2003

Data flow and control flow are separate

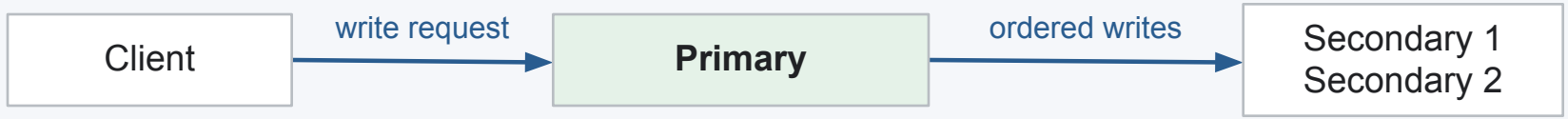
After metadata lookup: send the bytes, then request the write.

DATA · pipeline to all replica buffers



Path follows network proximity; it can reach a secondary first.

CONTROL · the primary decides the write order



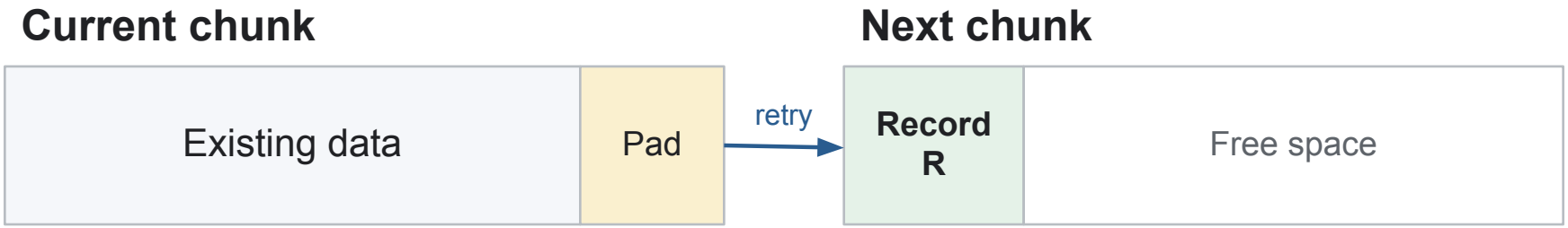
Secondaries apply → ACK primary → reply to client.

Buffering the bytes does not apply the write.

Source: Sections 3.1–3.2 and Figure 2 (adapted), Ghemawat et al., SOSP 2003

Record append: atomic, at least once

Client supplies a record; GFS chooses the offset.



If R does not fit: pad this chunk, then append R to the next.

Successful append

Complete record, at least once

Failures and retries

May leave duplicates or record fragments

Checksums validate records; unique IDs help detect duplicates.

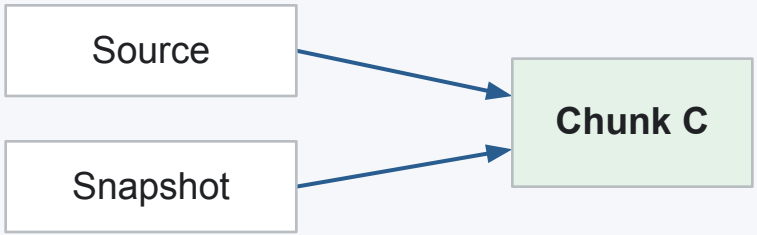
Source: Sections 2.7.2 and 3.3, Ghemawat et al., SOSP 2003

Snapshot uses copy-on-write

Fast copies of files or directory trees.

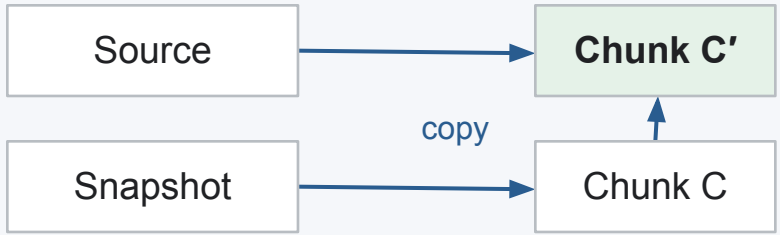
Revoke leases → log operation → copy metadata

Immediately after snapshot



Both files share the same chunks.

First write to the source



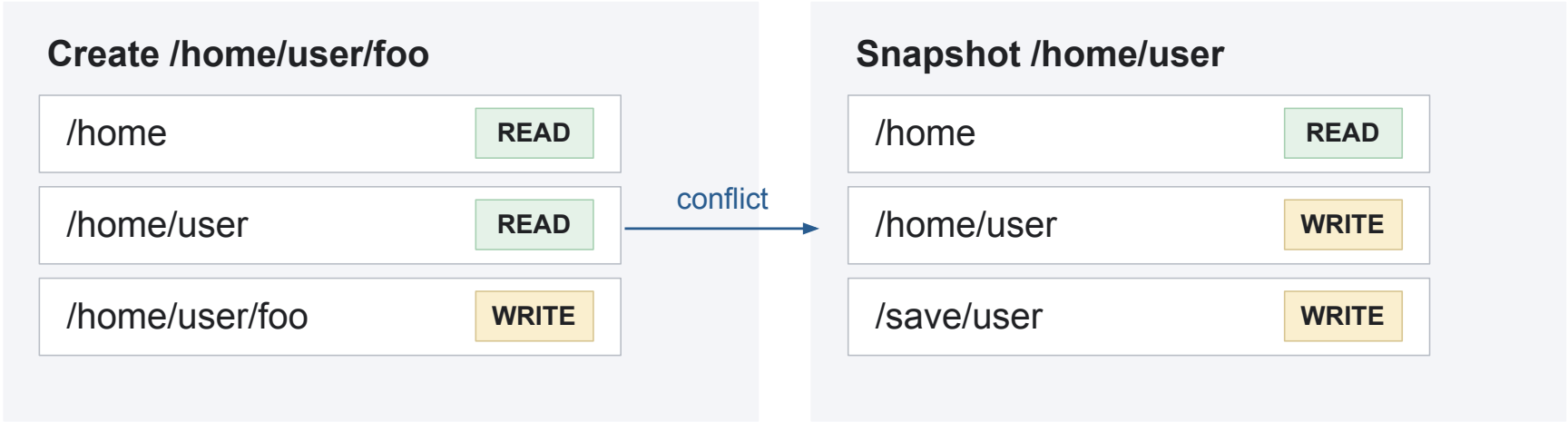
Source uses C′; snapshot keeps C.

Copy C locally before granting a new write lease.

Source: Section 3.4, Ghemawat et al., SOSP 2003

Namespace locks serialize conflicting operations

The master locks pathnames, not the entire filesystem.



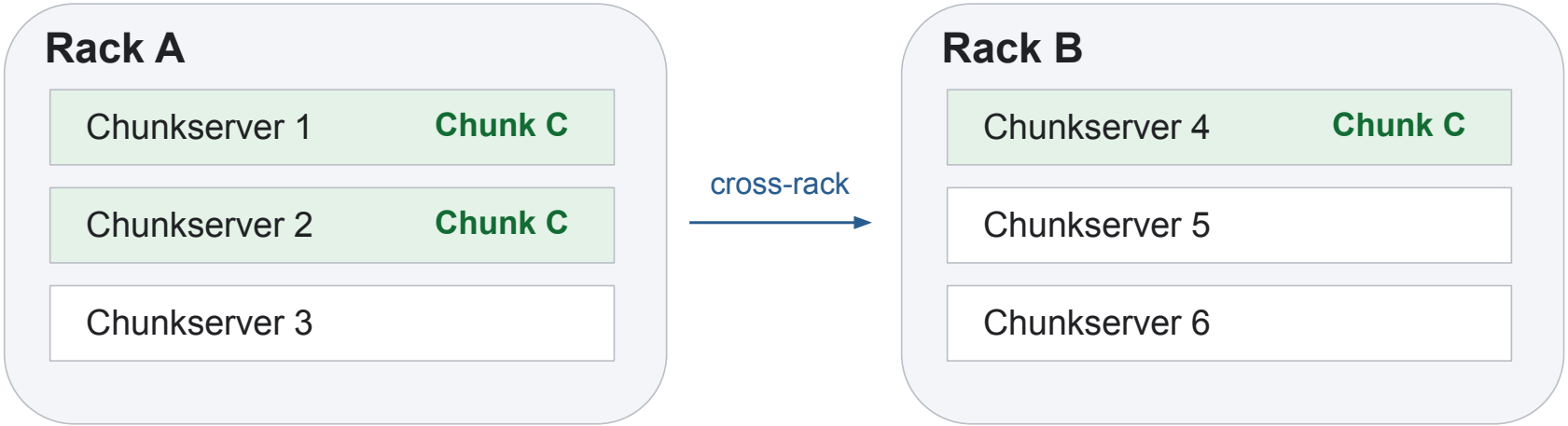
Only operations with conflicting locks must wait.

Different files can be created concurrently in the same directory.

Source: Section 4.1, Ghemawat et al., SOSP 2003

Replica placement spans racks

Three replicas by default; placement must tolerate more than machine failures.



Benefit: a rack outage need not lose the chunk.

Cost: writes send data across rack boundaries.

Replication improves reliability, availability, and read bandwidth.

Source: Section 4.2, Ghemawat et al., SOSP 2003

The master maintains replica placement

Creation, repair, and rebalancing address different conditions.

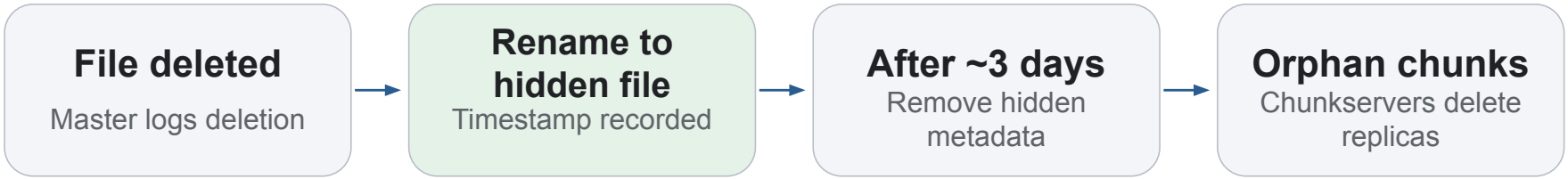
Activity	When?	Master policy
Creation	New chunk is needed	Choose low-use servers; avoid concentrated new writes; spread racks
Re-replication	Replica count falls below target	Clone from healthy copy; prioritize scarce replicas and blocked clients
Rebalancing	Regular background scan	Move replicas to balance disk usage and traffic

A chunk with one surviving replica receives higher repair priority than one with two.

Source: Section 4.3, Ghemawat et al., SOSP 2003

Deletion is lazy, not immediate

GFS reclaims storage during regular background garbage collection.



Why it helps
No fragile distributed delete protocol; accidental deletion can be undone temporarily.

Tradeoff

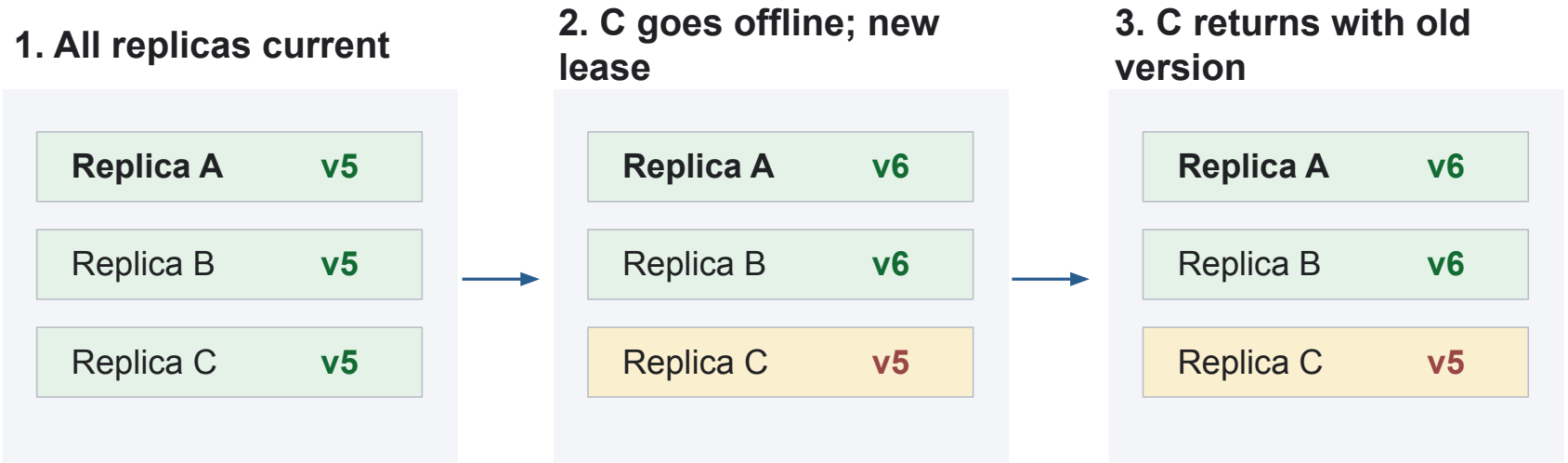
Disk space is not immediately reusable.

Unreferenced replicas become garbage, even after partial failures.

Source: Section 4.4, Ghemawat et al., SOSP 2003

Chunk versions identify stale replicas

A chunkserver can miss mutations while it is unavailable.



The master excludes stale copies from writes and location replies.

Version numbers advance when a new lease is granted, not on every write.

Source: Section 4.5, Ghemawat et al., SOSP 2003

Availability uses recovery and replication

Failures are routine, so GFS restores service and redundancy automatically.

Fast recovery

Master and chunkservers restart from persistent state in seconds.
Clients reconnect and retry.

Chunk replication

Chunks are replicated across racks.
The master clones healthy replicas when some become unavailable.

Master replication

Operation log and checkpoints are replicated remotely.
A new master can restart elsewhere.

Shadow masters offer read-only metadata access, but may lag the primary.

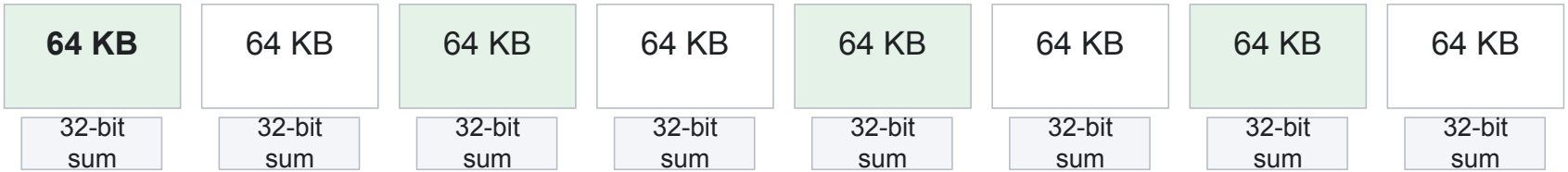
A single active master still controls all mutations.

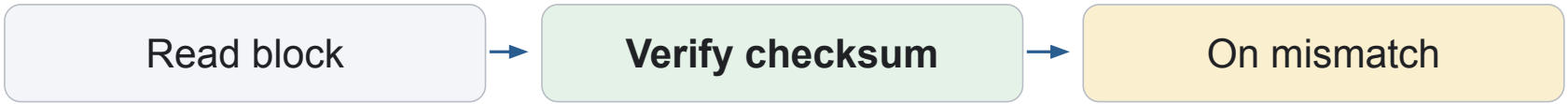
Source: Sections 5.1–5.1.3, Ghemawat et al., SOSP 2003

Checksums detect silent data corruption

Each 64 MB chunk is split into 64 KB blocks, each with a 32-bit checksum.

Selected blocks from one chunk (1,024 total)





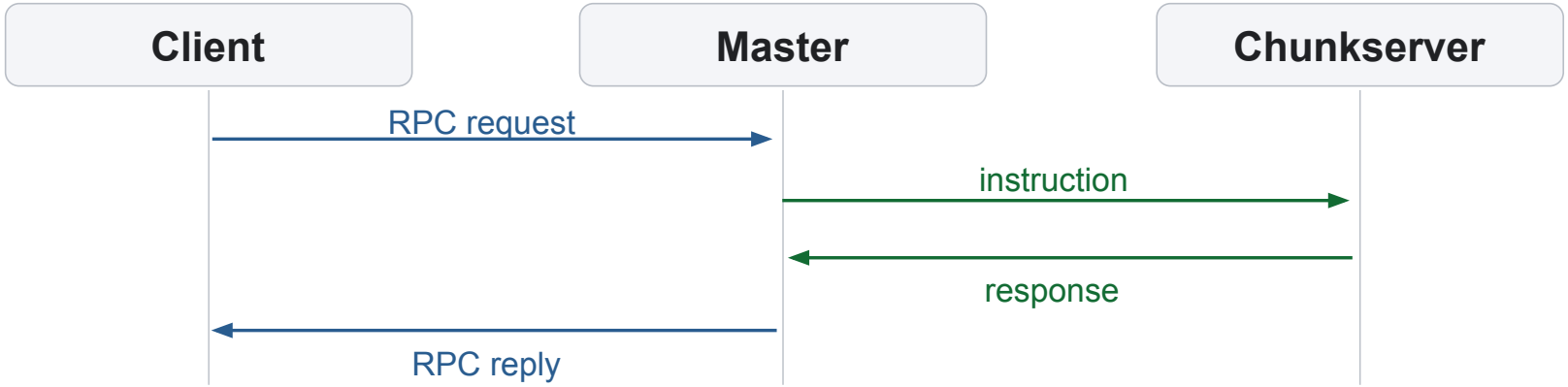
Report error → read healthy replica → replace corrupted copy.

Replica contents may legally differ; each server validates its own data.

Source: Section 5.2, Ghemawat et al., SOSP 2003

Diagnostic logs reconstruct failures

Distributed bugs are often transient and hard to reproduce.



Operation log: durable metadata changes

Diagnostic log: RPCs and server events

RPC traces help explain interactions across many machines.

Source: Section 5.3, Ghemawat et al., SOSP 2003

Micro-benchmarks: a controlled cluster

Measure aggregate throughput as the number of clients increases.

16 clients ↔ **16 chunkservers** + **1 master (+ 2 replicas)**

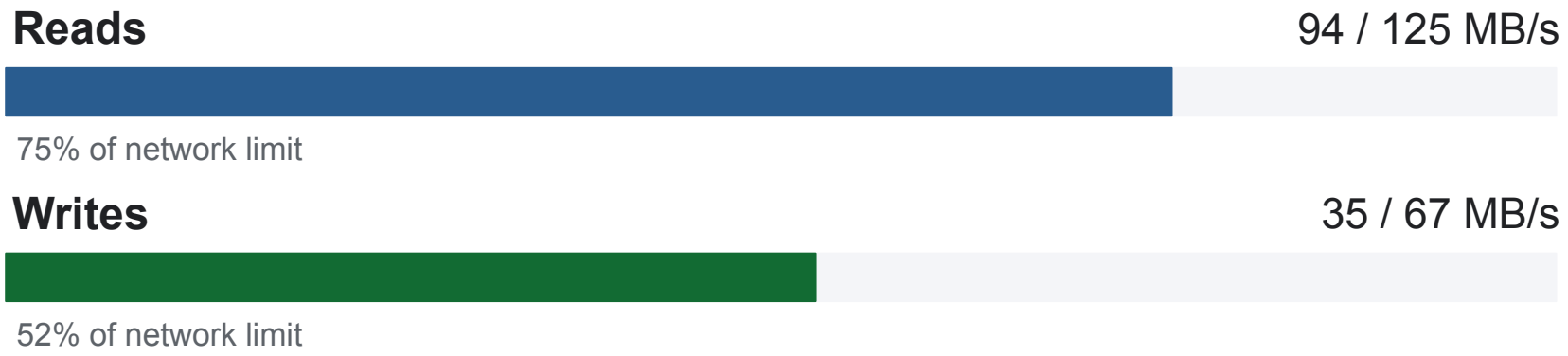
Workload	What each client does
Read	Read 4 MB ranges from a 320 GB file set
Write	Write 1 GB to a separate new file (1 MB at a time)
Record append	Append records concurrently to one shared file

100 Mbps per machine; 1 Gbps link between the two switches.

Source: Section 6.1, Ghemawat et al., SOSP 2003

Reads scale; writes pay for replication

Aggregate throughput at 16 clients (Figure 3).



Record append	4.8 MB/s at 16 clients; all append to the same last chunk.

Pipelined writes are slower; single-file appends form a hotspot.

Source: Sections 6.1.1–6.1.3 and Figure 3, Ghemawat et al., SOSP 2003

Real-world clusters are large; master state is small

Two representative GFS deployments at Google.

Metric	Cluster A	Cluster B
Workload	Research / development	Production data
Chunkservers	342	227
Used disk (including replicas)	55 TB	155 TB
Number of files	735 K	737 K
Number of chunks	992 K	1,550 K
Master metadata	48 MB	60 MB

Tens of MB of master metadata can track hundreds of TB of storage.

The used disk total includes three replicas for most files.

Source: Section 6.2 and Table 2, Ghemawat et al., SOSP 2003

Real traffic is read-heavy

Last-minute observations from two production-scale clusters.

Last-minute rate	Cluster A	Cluster B
Read throughput	**583 MB/s**	**380 MB/s**
Write throughput	1 MB/s	101 MB/s
Master requests	325 ops/s	533 ops/s

The master handles hundreds of metadata operations per second, without being a bottleneck.

Aggregate read bandwidth is high; demands vary by workload.

Source: Section 6.2.3–6.2.4 and Table 3, Ghemawat et al., SOSP 2003

Replica repair reduces the risk window

Chunkserver failures trigger prioritized cloning of missing replicas.

One chunkserver failed

~15,000 chunks / 600 GB

23.2 min

All missing replicas restored

Two chunkservers failed

266 chunks left with only one replica

< 2 min

Each restored to at least two replicas

At-risk chunks are repaired before less urgent replica losses.

Source: Section 6.2.5, Ghemawat et al., SOSP 2003

Observed workloads validate the design assumptions

Reads and writes are shaped by large files and append-heavy processing.

Read sizes

Bimodal

Small random (<64 KB) and large sequential (>512 KB) reads.

Transferred bytes

Large operations

Operations >256 KB account for most transferred bytes.

Overwritten bytes

Almost zero

<0.0001% in X; 0.05% in Y, often due to retries.

Around 64–66% of master requests find chunk replica locations.

Append-heavy and large-transfer behavior supports GFS specialization.

Source: Sections 6.3.2–6.3.4 and Tables 4–6, Ghemawat et al., SOSP 2003

Deployment exposed problems below the file system

Many practical failures came from disks and Linux kernel behavior.

Problem	Effect	Response
Disk / IDE mismatch	Silent data corruption	Checksums + kernel fixes
Linux 2.2 fsync()	Expensive operation-log flushes	Workaround, then Linux 2.4
Linux mmap() locking	Unexpected timeouts	Use pread() instead

Broader R&D use also created new requirements for permissions and quotas.

Source: Section 7, Ghemawat et al., SOSP 2003

GFS chooses different tradeoffs from prior systems

Its mechanisms were known; the workload-driven combination was distinctive.

Design dimension	Examples in prior systems	GFS choice
Coordination	Distributed metadata in some designs	Single master
Client data caching	AFS and several prior systems	No client-side file data cache
Redundancy	RAID / parity approaches	Simple chunk replication
File-system interface	More general POSIX-style features	Append + snapshot; relaxed consistency

GFS optimizes for large-scale data processing, not generality.

Source: Section 8, Ghemawat et al., SOSP 2003

GFS: workload-driven simplicity

The system combines simple coordination with resilient, high-throughput storage.

Assume failures

Monitor components
Replicate chunks
Recover automatically

Optimize for workload

Large files and chunks
Append-focused APIs
Relaxed consistency

Keep master off data path

Metadata only at master
Chunk leases for ordering
Direct, pipelined transfers

A centralized master can scale when data transfer bypasses it.

Source: Section 9, Ghemawat et al., SOSP 2003

Thank you